Case Study: Simple JS Pokedex App

Overview:

Simple JS App (Pokedex) is a lightweight web application built with **HTML, CSS, and JavaScript**. It fetches data from the public **PokeAPI** to display information (images, stats, etc.) for the first 9 generations of Pokémon.

Users can:

- Browse Pokémon in a list
- Click a button for a Pokémon to open a modal with detailed info
- Search by name via a dropdown / search bar
- Toggle "fancy mode" to switch styling or image sources (sprite vs official artwork)

The goal was to practice working with external APIs, dynamic DOM updates, modals, and UI interactivity in vanilla JS (with help from jQuery, Bootstrap, and polyfills).

p

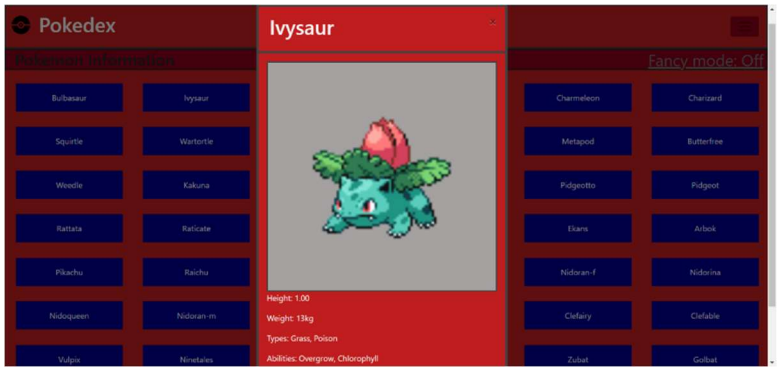
Problem / Motivation

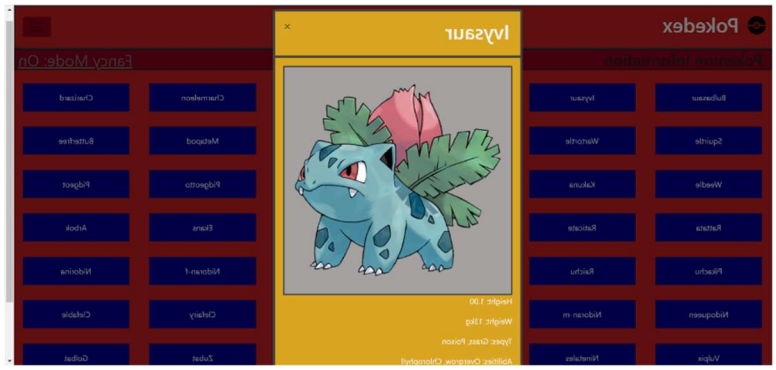
- • This was my introduction into **fetching API data**, handling asynchronous responses (Promises), and building a dynamic UI without relying on heavy frameworks.
- • Many example projects fetch from simple "dummy" endpoints; I wanted something more real-world and visually engaging (hence Pokémon).
- • I also wanted to experiment with UX touches like modal overlays, conditional styling ("fancy mode"), and graceful handling when no results are found.
-

Solution

The app solves that by:

- Fetching data for first 151 Pokémon from PokeAPI
- Using Dynamic endpoints to expand beyond the original 151 Pokamon
- Rendering a list of buttons (one per Pokémon).
- On click, opening a modal overlay with relevant info (image, stats, type, etc.).
- Providing a search dropdown / input to filter by name. If exactly one match, show the modal; if multiple, list filtered results; if none, show a “no results” modal.
- A “fancy mode” toggle that (1) switches to higher-quality official artwork images, (2) changes the modal color scheme. (Because official artwork is heavier, switching to it on demand gives users control over performance vs visuals.)

1) 

2) 

This provides a more interactive and polished experience relative to a barebones fetch-and-list example.

Implementation & Technical Choices

Tech Stack & Libraries

- **HTML / CSS / JavaScript** (vanilla)
- **jQuery** — to simplify DOM manipulation and event binding
- **Bootstrap** — for layout, modal styling, responsive UI
- **Fetch API & Promises** (with polyfills for compatibility)

Architecture & Components

- **Data layer**: A module to fetch Pokémon data (list + individual details).
- **UI layer**:
    - A function to build the list of Pokémon buttons.
    - A modal manager to open, populate, and close modals.
    - A search handler that filters names and triggers UI updates.
- **State & mode management**: A flag for "fancy mode" tracked globally, influencing which image URL to use and which CSS classes to apply.
- **Error / edge cases**: Handling API failures, no search results, multiple matches, etc.

Design Decisions & Trade-offs

- **On-demand artwork**: Official artwork images are larger, so I decided not to load them by default; instead "fancy mode" lets users opt in.
- **jQuery + vanilla vs pure modern JS frameworks**: Chose jQuery / vanilla to keep dependencies minimal and emphasize DOM fundamentals.
- **Modal-based detail view**: Instead of navigating to a separate page, I used modals to reduce context switching and keep the user within one page.
- **Simple state management**: Because the app is small, I avoided introducing a full state library or routing.

Results & Learnings

What worked well

- The UI feels responsive and interactive: clicking, searching, toggling modes all happen fluidly.

- The modular separation (fetching, UI, state) made it easier to reason about parts of the code.
- The “fancy mode” feature gave a nice UX twist and allowed toggling between performance / visuals.
- I deepened my familiarity with asynchronous JS, DOM manipulation, and handling edge cases in UI.

Challenges & limitations

- Loading all Pokémon in a generation’s details or large images can slow performance on weak devices or slow networks.
- No caching or incremental loading (e.g. paging) — everything is fetched upfront (or in bulk).
- Search is relatively simple (string matching) and not fuzzy (typos won’t match).
- Accessibility (a11y) and keyboard navigation were not deeply addressed.
- No unit tests or automated testing coverage yet.

Next Steps / Future Enhancements

Possible improvements or expansions:

- **Lazy loading / pagination**: Instead of loading all pokemon details at once, load in batches or on demand.
- **Caching & localStorage**: Cache fetched Pokémon data so repeating requests are avoided.
- **Advanced search / fuzzy matching**: Allow partial matches, typo tolerance, filtering by type or stats.
- **Accessibility enhancements**: Keyboard navigation, ARIA roles, focus management.
- **Testing**: Add unit tests (e.g. with Jest) or end-to-end tests (e.g. with Cypress).
- **Dark mode / styling themes**: More UI theming options.
- **Performance optimization**: Use image lazy loading, optimize asset sizes, etc.

Conclusion

The **Simple JS Pokedex App** is a solid example project that bridges fetch-based API consumption with meaningful UI interaction — all in a lightweight, dependency-light stack. Through it, you can learn how to manage asynchronous data, update the DOM dynamically, and shape a user experience (modals, toggles, error states).

Duration: Two Weeks. It took me about on week, to decide on the API I wanted to use, the layout and familiarize myself with JavaScript and modals. The second week was spent cleaning up my code and the layout, while added extra features.

Credits:

Role: Lead Developer

Tutor: Jesus Diaz

Mentor: John Akhilomen